TRAINING REFERENCE

Databricks Fundamental

Reference guide for the one-day course

Altkom Akademia
Edition: October 2026

Contents

MODULE 01	Platform and workspace	3
MODULE 02	Spark architecture	5
MODULE 03	ELT and ingestion	6
MODULE 04	Delta Lake	9
MODULE 05	Orchestration with Lakeflow Jobs	11
MODULE 06	Unity Catalog	13
MODULE 07	AI/BI dashboards and Genie Agents, optional	15
APPENDIX A	Connecting to ADLS Gen2	16
APPENDIX B	SQL quick reference	17
APPENDIX C	Documentation	18

MODULE 01

Platform and workspace

Lakehouse architecture

A lakehouse combines the open, low-cost storage of a data lake with the reliability and governance of a data warehouse. Databricks builds it from three layers.

Layer	Technology	Purpose
Storage	Delta Lake on cloud object storage	ACID transactions, versioning, schema enforcement
Compute	Spark clusters, SQL warehouses, serverless compute	Batch, streaming, machine learning, SQL analytics
Governance	Unity Catalog	Access control, lineage, discovery

Compute options

Type	Typical use	Behaviour
All-purpose cluster	Interactive notebooks and development	Runs until terminated; billed while running
Job cluster	Scheduled and automated jobs	Created for a run and terminated when it ends
SQL warehouse	SQL editor and BI tools over JDBC or ODBC	Suspends when idle; optimised for concurrent queries
Serverless compute	Notebooks, jobs and SQL without cluster management	Starts in seconds; billed per second of use

For new development, serverless compute is usually the right default: it starts in seconds, needs no configuration and costs nothing while idle.

Storage paths: volumes, not DBFS

	DBFS (legacy)	Unity Catalog volume
Path	`dbfs:/…`, `dbfs:/mnt/…`	`/Volumes/<catalog>/<schema>/<volume>`
Governance	None; visible to every workspace user	Unity Catalog permissions, audit, lineage
New workspaces	Not available	Default

> DBFS root and mounts are deprecated. Azure Databricks accounts created after 18 December 2025 have no access to them, nor to the Hive metastore or no-isolation shared compute. From 8 October 2026 every new workspace, in any account, is created without these features and without Databricks Runtime versions older than 13.3 LTS. Existing workspaces keep them. Use volume paths in new code.

Magic commands

Command	Effect
`%sql`	Run the cell as SQL
`%python`	Run the cell as Python
`%sh`	Run shell commands on the driver
`%pip install <package>`	Install a library for this notebook session
`%run ./path/notebook`	Run another notebook in the current scope
`%fs ls /Volumes/…`	List files in a volume
`%skip`	Skip the cell when running the whole notebook

Genie Code, the notebook assistant

Genie Code, formerly Databricks Assistant, writes, explains and fixes code in notebooks and the SQL editor. It reads Unity Catalog metadata, so its suggestions refer to the tables and columns in your catalog.

Action	How
Inline prompt	`Cmd+I` on macOS, `Ctrl+I` on Windows
Explain code	`/explain`
Fix failing code	`/fix`, or **Diagnose error** on the failed cell
Generate documentation	`/doc`
Suggest a faster version	`/optimize`
Reformat	`/prettify`

Genie Code can also carry out multi-step tasks, such as exploring a table and writing the code, and asks for your approval before it runs anything. It belongs to the Genie family together with Genie Agents (natural-language questions over chosen tables, M07) and Genie One (the chat interface for business users).

> Review generated code before running it, particularly code that writes or deletes data.

Widgets (BONUS)

```
dbutils.widgets.text("env", "dev", "Environment")
dbutils.widgets.dropdown("format", "delta", ["delta", "parquet", "csv"])
env = dbutils.widgets.get("env")
dbutils.widgets.removeAll()
```

MODULE 02

Spark architecture

Driver and executors

Component	Role
Driver	Runs your code, builds the execution plan, schedules tasks and collects results. One per cluster.
Executor	Runs tasks in parallel, each on one partition held in memory or on disk. Several per cluster.
Partition	The slice of data processed by a single task. Default target size 128 MB.

Transformations and actions

Concept	Meaning	Examples
Transformation	Lazy: defines a new DataFrame without computing it	`filter`, `select`, `join`, `groupBy`
Action	Triggers computation and returns or writes a result	`count()`, `show()`, `write`, `collect()`
Narrow transformation	Works within each partition; no data moves	`filter`, `select`
Wide transformation	Redistributes data across partitions (a shuffle)	`groupBy`, `join`, `distinct`

DAG, stages and tasks (BONUS)

Spark turns a chain of transformations into a directed acyclic graph (DAG). The graph is divided into stages at every shuffle, and each stage runs as one task per partition. Shuffles move data across the network between executors and are the most expensive part of most jobs.

Catalyst and adaptive query execution (BONUS)

Feature	Runs at	What it does
Catalyst optimizer	Planning time	Rewrites the logical plan into a physical plan: predicate pushdown, column pruning, constant folding
Adaptive query execution (AQE)	Run time	Re-plans using real shuffle statistics: merges small partitions, switches join strategies, splits skewed partitions

AQE is enabled by default in Databricks Runtime. The DataFrame API and `spark.sql()` compile to the same plan, so choose whichever reads better.

MODULE 03

ELT and ingestion

Reading files

```
# CSV: an explicit schema avoids a second pass over large files
df = (spark.read.format("csv")
      .option("header", True)
      .option("sep", ",")
      .option("nullValue", "NULL")
      .schema(my_schema)
      .load("/Volumes/<catalog>/<schema>/<volume>/customers.csv"))

# JSON Lines (one record per line, the default; the course data uses it)
df = spark.read.json("/Volumes/…/orders/")
# Only for a single JSON document spread over several lines
df = spark.read.option("multiLine", True).json("/Volumes/…/orders.json")

# Parquet and Delta carry their own schema
df = spark.read.parquet("/Volumes/…/products/")
df = spark.read.table("catalog.schema.table")
```

```
-- The same JSON read in SQL (read_files)
SELECT * FROM read_files('/Volumes/<catalog>/<schema>/<volume>/orders/', format => 'json');
```

Exploring a DataFrame

Method	Output
`display(df)`	Interactive table; charts and a data profile are available from the **+** menu
`df.show(5, truncate=False)`	Plain-text preview
`df.printSchema()`	Column names and types
`df.describe()`	Count, mean, standard deviation, minimum and maximum
`df.summary()`	The same as `describe()`, plus the 25th, 50th and 75th percentiles

Common transformations

Operation	Syntax
Select columns	`df.select("col1", "col2")`
Add or change a column	`df.withColumn("new", col("x") * 2)`
Rename a column	`df.withColumnRenamed("old", "new")`
Filter rows	`df.filter(col("age") > 18)`
Conditional value	`when(col("x") > 0, "pos").otherwise("neg")`
Drop a column	`df.drop("col")`
Change a type	`col("id").cast("integer")`
Safe type change	`expr("try_cast(x AS INT)")`; SQL `try_cast(x AS INT)` returns NULL instead of a CAST_INVALID_INPUT error
Write to Delta	`df.write.mode("overwrite").saveAsTable("catalog.bronze.orders")`; SQL `CREATE TABLE ... AS SELECT`
Remove duplicates	`df.dropDuplicates(["col1", "col2"])`
Sort	`df.orderBy(col("date").desc())`
Handle nulls	`df.fillna({"col": 0})`, `df.dropna()`

Aggregation

```
from pyspark.sql.functions import count, sum, avg, col

(df.groupBy("region")
   .agg(count("*").alias("orders"),
        sum("amount").alias("revenue"),
        avg("amount").alias("avg_order"))
   .filter(col("orders") > 100))        # equivalent of SQL HAVING
```

Joins (BONUS)

Type	Rows returned
`inner`	Rows that match in both DataFrames
`left`	All rows from the left side, with matches from the right or nulls
`right`	All rows from the right side, with matches from the left or nulls
`full`	All rows from both sides
`cross`	Every combination of rows

```
orders.join(customers, orders.customer_id == customers.id, "left")
```

Temporary views

```
df.createOrReplaceTempView("orders_view")
spark.sql("SELECT region, SUM(amount) FROM orders_view GROUP BY region").show()
```

Nested JSON (BONUS)

```
from pyspark.sql.functions import explode, get_json_object, col

df.withColumn("item", explode(col("items")))                                # array to rows
df.withColumn("city", get_json_object(col("json_str"), "$.address.city"))
```

MODULE 04

Delta Lake

Delta Lake stores data as Parquet files plus a transaction log in `_delta_log/`. Every change is recorded in the log, which is what makes ACID transactions, time travel and schema enforcement possible.

ACID guarantees

Property	Meaning
Atomicity	A write either completes fully or leaves no trace
Consistency	Schema and constraints are checked on every write
Isolation	Concurrent readers and writers do not see each other's partial work
Durability	Committed data survives failures

Changing data

```
INSERT INTO catalog.schema.orders VALUES (1001, 'C001', 99.99, current_date());

UPDATE catalog.schema.orders SET status = 'shipped' WHERE order_id = 1001;

DELETE FROM catalog.schema.orders WHERE status = 'cancelled';

-- Upsert: update matches, insert new rows, delete rows missing from the source
MERGE INTO target t
USING source s ON t.id = s.id
WHEN MATCHED THEN UPDATE SET t.val = s.val
WHEN NOT MATCHED THEN INSERT *
WHEN NOT MATCHED BY SOURCE THEN DELETE;
```

Time travel

```
DESCRIBE HISTORY my_table;

SELECT * FROM my_table VERSION AS OF 5;
SELECT * FROM my_table TIMESTAMP AS OF '2026-01-15 08:00:00';

RESTORE TABLE my_table TO VERSION AS OF 5;
```

```
df = spark.read.option("versionAsOf", 5).table("my_table")
```

VACUUM

```
VACUUM my_table DRY RUN;             -- list the files that would be removed
VACUUM my_table;                     -- remove files older than 7 days (the default)
VACUUM my_table RETAIN 168 HOURS;    -- the same, with the retention stated explicitly
```

> VACUUM removes old data files, not the `_delta_log`. Files it removes cannot be recovered, and table versions that depended on them can no longer be reached with time travel or `RESTORE`. Keep the retention at 7 days or more on tables that are read concurrently.

Liquid clustering and predictive optimization

```
CREATE TABLE my_table (order_id STRING, order_date DATE, amount DOUBLE)
CLUSTER BY (order_date);                    -- cluster by columns you filter on
ALTER TABLE my_table CLUSTER BY AUTO;       -- let Databricks choose the keys
OPTIMIZE my_table;                          -- compact and cluster new data
```

Liquid clustering is the recommended data layout for new tables and replaces partitioning and Z-ordering. With predictive optimization, Databricks runs maintenance such as `OPTIMIZE` and `VACUUM` on Unity Catalog managed tables for you. `OPTIMIZE ... ZORDER BY (col)` remains for older tables without clustering.

Managed and external tables

	Managed table	External table
Data location	Storage managed by Unity Catalog	A cloud path you specify (`abfss://`, `s3://`, `gs://`)
`DROP TABLE`	Removes metadata and data files	Removes metadata only; the files remain
Use when	By default	Data must stay in an existing location

MODULE 05

Orchestration with Lakeflow Jobs

Concepts

Term	Meaning
Job	A scheduled or on-demand unit made of one or more tasks
Task	One step: a notebook, Python script, SQL query, pipeline or similar
Dependency	The order in which tasks run; the tasks of a job form a DAG
Repair run	Re-runs only the failed tasks and the tasks downstream of them
Job cluster	Compute created for a run and removed when it finishes

Triggers

Trigger	Runs the job
Scheduled	On a cron schedule
File arrival	When new files appear in a storage location
Continuous	Again as soon as the previous run ends
Manual	On request, from the UI or the API

Cron expressions

Databricks uses the Quartz format, which begins with a seconds field.

```
seconds minutes hours day-of-month month day-of-week
   0      30      2        *          *        ?        every day at 02:30
   0       0      8        ?          *     MON-FRI     weekdays at 08:00
   0       0      0        1          *        ?        first day of each month at midnight
```

Passing values between tasks

```
# Read a job or task parameter (the default is used outside a job)
dbutils.widgets.text("source_table", "samples.nyctaxi.trips")
source_table = dbutils.widgets.get("source_table")

# Task "validate": publish a value for downstream tasks
dbutils.jobs.taskValues.set(key="row_count", value=row_count)

# Task "transform": read it; debugValue is used when the notebook runs outside a job
row_count = dbutils.jobs.taskValues.get(taskKey="validate", key="row_count", debugValue=-1)
```

Job parameters apply to every task. If a job parameter and a task parameter share a key, the job parameter wins.

Lakeflow Spark Declarative Pipelines (BONUS)

Lakeflow Spark Declarative Pipelines, formerly Delta Live Tables, define tables declaratively; Databricks works out the dependencies and the refresh order. Create one in **Jobs & Pipelines**, then **Create**, **ETL pipeline**.

Object	Behaviour
Streaming table	Ingests incrementally, processing only data that is new since the last update
Materialized view	Stores the result of a query and refreshes it on each pipeline update
View	A named query used inside the pipeline; not stored

```
CREATE OR REFRESH STREAMING TABLE bronze_orders
AS SELECT * FROM STREAM read_files('/Volumes/<catalog>/<schema>/<volume>/orders/', format =>
'json');

CREATE OR REFRESH MATERIALIZED VIEW silver_orders
AS SELECT order_id, customer_id, total_amount FROM bronze_orders WHERE total_amount > 0;
```

Tables in the same pipeline are referenced by name; the old `LIVE.` prefix is not needed.

Monitoring failed runs (trainer demo)

```
SELECT *
FROM system.lakeflow.job_run_timeline
WHERE result_state = 'FAILED'
  AND period_start_time >= current_timestamp() - INTERVAL 7 DAYS;
```

Querying system tables requires access to the `system` catalog.

MODULE 06

Unity Catalog

Object hierarchy

```
Metastore           one per region, shared by the workspaces in that region
└── Catalog         for example dev, prod, sandbox
    └── Schema      a logical group of objects, comparable to a database
        └── Table, view, volume, function
```

Refer to objects by their full three-part name, `catalog.schema.table`, for example `main.retail.orders`.

Granting privileges

Privileges must be granted at each level, starting from the catalog.

```
GRANT USE CATALOG ON CATALOG main              TO `retailhub_analysts`;
GRANT USE SCHEMA  ON SCHEMA  main.retail       TO `retailhub_analysts`;
GRANT SELECT      ON TABLE   main.retail.orders TO `retailhub_analysts`;

SHOW GRANTS ON TABLE main.retail.orders;

REVOKE SELECT ON TABLE main.retail.orders FROM `retailhub_analysts`;
```

Privilege	Granted on	Allows
`USE CATALOG`	Catalog	Opening the catalog; required first
`USE SCHEMA`	Schema	Opening the schema
`SELECT`	Table, view, or a schema for all its tables	Reading data
`MODIFY`	Table	Inserting, updating and deleting data
`CREATE TABLE`	Schema	Creating tables
`MANAGE`	Any securable object	Granting and revoking privileges on that object
`ALL PRIVILEGES`	Any securable object	Most privileges, but not `MANAGE`

Only the owner of an object, a metastore admin or a holder of `MANAGE` can grant privileges on it. `ALL PRIVILEGES` alone is not sufficient.

Groups

Grant privileges to groups rather than to individual users. Unity Catalog accepts only account groups.

	Account group	Workspace-local group
Created in	The account console, or workspace settings under Identity and access	The Workspace Groups API, or SQL `CREATE GROUP`
Can receive Unity Catalog privileges	Yes	No; the grant fails with `PRINCIPAL_DOES_NOT_EXIST`

> SQL `CREATE GROUP` completes without error but creates a workspace-local group. Create groups in the account console or in the workspace settings instead.

External storage (BONUS)

Object	Purpose
Storage credential	Holds the cloud identity, such as an Azure Access Connector, used to reach storage
External location	Links a storage path to a credential and controls `READ FILES` and `WRITE FILES`
External table	A table whose files are stored in an external location

MODULE 07

AI/BI dashboards and Genie Agents, optional

AI/BI dashboards

Dashboards present Unity Catalog data as charts, counters and tables, with date pickers, drop-down filters and cross-filtering between widgets. A published dashboard has a shareable link, does not require viewers to run queries, can be emailed on a schedule and can be embedded in other applications.

Genie Agents

A Genie Agent (formerly Genie space) answers questions asked in plain language by generating and running SQL against the Unity Catalog tables configured for it. Create one from **Genie Agents** in the sidebar; it needs a pro or serverless SQL warehouse and partner-powered AI features switched on in the workspace. It keeps the context of a conversation, so follow-up questions work, and answers can be saved as dashboard widgets. Genie suits exploratory, self-service questions; it is less reliable for multi-step logic or reports with strict formatting.

Choosing a tool

Need	Tool
A reviewed KPI dashboard refreshed every day	AI/BI dashboard
An ad-hoc question, such as last month's ten best-selling products	Genie Agent
Exploring data before building a dashboard	SQL editor, which has its own visualizations

APPENDIX A

Connecting to ADLS Gen2

Step	Where	Action
1	Azure portal, Databricks managed resource group	Copy the resource ID of the Access Connector (`Microsoft.Databricks/accessConnectors/<name>`)
2	Storage account, Access control (IAM)	Assign **Storage Blob Data Contributor** to the Access Connector's managed identity
3	Catalog Explorer, External data, Credentials	Create a storage credential with the Access Connector resource ID
4	Catalog Explorer, External locations	Create an external location from the `abfss://` URL and the credential
5	External location, Permissions	Grant `READ FILES`, `WRITE FILES` or `CREATE EXTERNAL TABLE` as required

```
SHOW EXTERNAL LOCATIONS;
LIST 'abfss://container@account.dfs.core.windows.net/';
```

> For read-only production sources, assign Storage Blob Data Reader instead of Contributor, and grant individual Unity Catalog privileges rather than `ALL PRIVILEGES`.

APPENDIX B

SQL quick reference

Delta Lake

```
DESCRIBE DETAIL my_table;
DESCRIBE HISTORY my_table;
ALTER TABLE my_table CLUSTER BY (col);
OPTIMIZE my_table;
VACUUM my_table RETAIN 168 HOURS;
RESTORE TABLE my_table TO VERSION AS OF 3;
SELECT * FROM my_table VERSION AS OF 2;
SELECT * FROM my_table TIMESTAMP AS OF '2026-01-15 08:00:00';
```

Unity Catalog

```
SHOW CATALOGS;
SHOW SCHEMAS IN my_catalog;
SHOW TABLES IN my_catalog.my_schema;
DESCRIBE TABLE EXTENDED my_catalog.my_schema.my_table;
SHOW GRANTS ON TABLE my_catalog.my_schema.my_table;
```

APPENDIX C

Documentation

Topic	Address
Databricks documentation	docs.databricks.com
Delta Lake	docs.delta.io
Apache Spark	spark.apache.org/docs/latest
Lakeflow Jobs	docs.databricks.com/en/workflows/index.html
Lakeflow Spark Declarative Pipelines	learn.microsoft.com/azure/databricks/ldp/
Unity Catalog	docs.databricks.com/en/data-governance/unity-catalog/index.html
Unity Catalog privileges	docs.databricks.com/en/data-governance/unity-catalog/manage-privileges/index.html
Storage credentials and external locations	docs.databricks.com/en/connect/unity-catalog/external-locations.html
Serverless compute	docs.databricks.com/en/compute/serverless.html
Adaptive query execution	docs.databricks.com/en/optimizations/aqe.html
Liquid clustering	learn.microsoft.com/azure/databricks/tables/clustering
Predictive optimization	learn.microsoft.com/azure/databricks/optimizations/predictive-optimization
AI/BI dashboards	docs.databricks.com/en/dashboards/index.html
Genie Agents	learn.microsoft.com/azure/databricks/genie-agents/
Genie Code	docs.databricks.com/en/genie-code/
Retirement of DBFS and other legacy features	docs.databricks.com/en/admin/account-settings/legacy-features.html